

گروه D

محمد امین حیدری 401170553

سینا محمدی 401110064

امیر حسین قاسمی پور 401106341

۱. هدف آزمایش

هدف این آزمایش، طراحی و پیاده‌سازی یک واحد محاسبه و منطق (ALU) برای اعداد مختلط با Verilog و سپس قرار دادن آن در یک کامپیوتر پایه با حافظه ۳۲ کلمه‌ای به صورت پاییلاین است. ماشین فرضی پیاده‌سازی شده، دستورات را از حافظه دستورالعمل خوانده و عملیات مختلط را روی اعداد مختلط ذخیره شده در حافظه داده اجرا می‌کند.

در نسخه توسعه‌یافته‌ی این پروژه، علاوه بر عملیات پایه، دو عملیات جدید به ALU اضافه شده است:

- تقسیم اعداد مختلط (با فرمول $(c+di)/(a+bi) = ((c^2+d^2) / (i(bc-ad) + (ac+bd)))$)
- مجذور اندازه عدد مختلط ($|z|^2 = a^2 + b^2$)

همچنین پروژه به گونه‌ای توسعه یافته است که روی برد آلترا DE2 قابل اجرا باشد، با نمایش نتایج بر روی LEDهای قرمز و سبز.

این پروژه از پنج بخش اصلی تشکیل شده است:

- پیمانه جمع و تفریق اعداد مختلط
- پیمانه ضرب اعداد مختلط
- پیمانه تقسیم اعداد مختلط
- پیمانه محاسبه مجذور اندازه
- واحد پاییلاین که دستورات را از حافظه خوانده و به صورت پاییلاین اجرا می‌کند

۲. محدودیت اصلی طراحی

مطابق با صورت آزمایش، در این طراحی نبایستی از واحدهای جمع‌کننده یا ضرب‌کننده بیش از یک بار استفاده شود. این بدان معناست که در کل طراحی تنها یک ضرب‌کننده حقیقی (mul)، یک جمع/تفریق‌کننده حقیقی (addsub)، و یک تقسیم‌کننده حقیقی (divider) قابل استفاده است.

این محدودیت اساسی‌ترین چالش طراحی است. به عنوان نمونه:

- ضرب دو عدد مختلط نیاز به چهار ضرب حقیقی و دو جمع/تفریق دارد.
- تقسیم دو عدد مختلط نیاز به شش ضرب حقیقی، سه جمع/تفریق و دو تقسیم دارد.

برای رعایت این محدودیت، یک ALU چندسیکلی طراحی شده است که با استفاده از یک ماشین حالت (FSM)، واحدهای مشترک را در سیکل‌های مختلف بین عملیات مختلف زمان-تقسیم می‌کند.

۳. ساختار کلی پروژه

پروژه از فایل‌های زیر تشکیل شده است:

- تعاریف ماکروها (عرض کلمه، دسترسی به بخش حقیقی و موهومی) — `macros.v`
- یک جمع/تفریق‌کننده حقیقی ۸ بیتی (تنها یک نمونه) — `addsub.v`
- یک ضرب‌کننده حقیقی ۸ بیتی (تنها یک نمونه) — `mul.v`
- یک تقسیم‌کننده حقیقی ۸ بیتی (تنها یک نمونه) — `divider.v`
- چندسیکلی که واحدهای بالا را زمان-تقسیم می‌کند — `alu.v`
- (PC) حافظه دستورالعمل و شمارنده برنامه — `inst_fetch.v`
- حافظه داده با دو پورت خواندن و یک پورت نوشتن — `memory.v`
- LED مازول سطح بالایی پایپلاین با خروجی‌های — `pipeline.v`
- تست‌بنچ‌های چهارگانه — `*_TB.v`
- مقادیر اولیه حافظه — `data/inst_mem.txt` و `data/initial_mem.txt`
- فایل پین‌اساینمنت برای برد — `pipeline_pins.qsf`

۴. تشریح فایل‌ها و کد

۴-۱. فایل `macros.v`

این فایل تعاریف اصلی پروژه را در بر دارد. عرض کلمه (WL) برابر ۸ بیت تعریف شده و یک عدد مختلط به صورت کلمه ۱۶ بیتی ذخیره می‌شود که ۸ بیت بالایی بخش حقیقی و ۸ بیت پایینی بخش موهومی است.

```
`define WL 8
`define complex [2*`WL-1:0]
`define Re(c) c[2*`WL-1:`WL]
`define Im(c) c[`WL-1:0]
`define sRe(c) $signed(`Re(c))
`define sIm(c) $signed(`Im(c))
```

۴-۲. فایل `addsub.v`

این فایل فقط شامل یک جمع‌کننده/تفریق‌کننده حقیقی ۸ بیتی است. ALU این پیمان را در سیکل‌های مختلف هم برای جمع/تفریق مختلط (یکبار برای بخش حقیقی و یکبار برای موهومی) و هم برای مراحل پایانی ضرب و تقسیم مختلط استفاده می‌کند.

```
module addsub (
    input signed [`WL-1:0] x,
    input signed [`WL-1:0] y,
```

```

    input                op,    // 0: add, 1: subtract
    output signed [`WL-1:0] s
);
    assign s = op ? (x - y) : (x + y);
endmodule

```

۴-۳. فایل mul.v

این فایل فقط يك ضرب‌کننده حقيقي ۸ بیتی است. خروجي به ۸ بیت کوتاه (truncate) می‌شود تا با عرض ذخیره‌سازي اعداد مختلط سازگار باشد.

```

module mul (
    input  signed [`WL-1:0] x,
    input  signed [`WL-1:0] y,
    output signed [`WL-1:0] p
);
    assign p = x * y;
endmodule

```

۴-۴. فایل divider.v (جدید)

این پیمانۀ يك تقسیم‌کننده حقيقي ۸ بیتی است که برای عملیات تقسیم مختلط افزوده شده است. تنها يك نمونه از آن در کل طراحی وجود دارد و ALU آن را در دو سیکل آخر تقسیم (D8 و D9) با ورودی‌های متفاوت استفاده می‌کند. در صورتی که مخرج صفر باشد، خروجي صفر می‌شود تا شبیه‌سازي ایمن بماند.

```

module divider (
    input  signed [`WL-1:0] n,
    input  signed [`WL-1:0] d,
    output signed [`WL-1:0] q
);
    assign q = (d == 0) ? 0 : n / d;
endmodule

```

۴-۵. فایل alu.v (هسته اصلي طراحی)

این پیمانۀ قلب طراحی است. ALU به صورت چندسیکلي طراحی شده و دقیقاً يك نمونه از mul، يك نمونه از addsub، و يك نمونه از divider را با كمك يك ماشین حالت ۲۰ حالتۀ زمان-تقسیم می‌کند.

کدگذاری عملیات (op) — اکنون ۳ بیتی:

op	عمل	تعداد سیکل
000	جمع مختلط	۲
001	تفریق مختلط	۲
010	ضرب مختلط	۵

op	عمل	تعداد سیکل
011	تقسیم مختلط	۹
100	مجذور اندازه	۳

کدگذاری عملیات با ثابت‌های localparam به صورت خوانا تعریف شده‌اند:

```
localparam OP_ADD    = 3'b000,
           OP_SUB    = 3'b001,
           OP_MUL    = 3'b010,
           OP_DIV    = 3'b011,
           OP_MAGSQ  = 3'b100;
```

حالت‌های ماشین حالت

```
localparam IDLE = 5'd0,
           AS1  = 5'd1, AS2 = 5'd2,
           M1   = 5'd3, M2  = 5'd4, M3 = 5'd5, M4 = 5'd6, M5 = 5'd7,
           D1   = 5'd8, D2  = 5'd9, D3 = 5'd10, D4 = 5'd11,
           D5   = 5'd12, D6  = 5'd13, D7 = 5'd14, D8 = 5'd15, D9 = 5'd16,
           Q1   = 5'd17, Q2  = 5'd18, Q3 = 5'd19;
```

رجیسترهای میانگاه

برای ذخیره نتایج بین سیکل‌ها، رجیسترهای زیر تعریف شده‌اند:

```
reg signed [`WL-1:0] t_ac, t_bd, t_ad, t_bc;    // اشتراکی بین
mul/div/magsq
reg signed [`WL-1:0] t_cc, t_dd;               // فقط برای تقسیم c2 و d2
reg signed [`WL-1:0] num_re, num_im, denom;    // میانگاه تقسیم
```

منطق ترکیبی مالتی‌پلکسر

این بخش ورودی‌های واحدهای اشتراکی را بر اساس حالت فعلی تنظیم می‌کند. در هر سیکل، حداکثر یک ضرب و یک جمع/تفریق و یک تقسیم انجام می‌شود:

```
always @(*) begin
    mul_x = 0; mul_y = 0;
    as_x  = 0; as_y  = 0; as_op = 0;
    div_n = 0; div_d = 1; // div_d=1 برای جلوگیری از تقسیم بر صفر
    case (state)
        // جمع / تفریق
        AS1: begin as_x = `sRe(a); as_y = `sRe(b); as_op = sub; end
        AS2: begin as_x = `sIm(a); as_y = `sIm(b); as_op = sub; end

        // ضرب: (a+bi)(c+di) = (ac-bd) + (ad+bc)i
```

```

M1 : begin mul_x = `sRe(a); mul_y = `sRe(b); end
M2 : begin mul_x = `sIm(a); mul_y = `sIm(b); end
M3 : begin mul_x = `sRe(a); mul_y = `sIm(b);
      as_x = t_ac; as_y = t_bd; as_op = 1'b1; end
M4 : begin mul_x = `sIm(a); mul_y = `sRe(b); end
M5 : begin as_x = t_ad; as_y = t_bc; as_op = 1'b0; end

// تقسیم: (a+bi)/(c+di) = ((ac+bd) + (bc-ad)i) / (c²+d²)
D1 : begin mul_x = `sRe(a); mul_y = `sRe(b); end
D2 : begin mul_x = `sIm(a); mul_y = `sIm(b); end
D3 : begin mul_x = `sRe(a); mul_y = `sIm(b);
      as_x = t_ac; as_y = t_bd; as_op = 1'b0; end
D4 : begin mul_x = `sIm(a); mul_y = `sRe(b); end
D5 : begin mul_x = `sRe(b); mul_y = `sRe(b);
      as_x = t_bc; as_y = t_ad; as_op = 1'b1; end
D6 : begin mul_x = `sIm(b); mul_y = `sIm(b); end
D7 : begin as_x = t_cc; as_y = t_dd; as_op = 1'b0; end
D8 : begin div_n = num_re; div_d = denom; end
D9 : begin div_n = num_im; div_d = denom; end

// مجذور اندازه: |a|² = Re(a)² + Im(a)²
Q1 : begin mul_x = `sRe(a); mul_y = `sRe(a); end
Q2 : begin mul_x = `sIm(a); mul_y = `sIm(a); end
Q3 : begin as_x = t_ac; as_y = t_bd; as_op = 1'b0; end
endcase
end

```

جدول زمان‌بندی ضرب مختلط (۵ سیکل)

سیکل	ضرب‌کننده	جمع/تفریق‌کننده	ثبت شدن در لبه بعد
M1	$\text{Re}(a) \times \text{Re}(b)$	بیکار	—
M2	$\text{Im}(a) \times \text{Im}(b)$	بیکار	$t_{ac} \leftarrow ac$
M3	$\text{Re}(a) \times \text{Im}(b)$	$ac - bd$	$t_{bd} \leftarrow bd$
M4	$\text{Im}(a) \times \text{Re}(b)$	بیکار	$t_{ad} \leftarrow ad$ ، $\text{Re}(\text{result}) \leftarrow ac - bd$
M5	بیکار	$ad + bc$	$t_{bc} \leftarrow bc$
بعد	—	—	$\text{Im}(\text{result}) \leftarrow ad + bc$ ، $\text{done} = 1$

جدول زمان‌بندی تقسیم مختلط (۹ سیکل)

این جدول نشان می‌دهد که چگونه فرمول تقسیم در نه سیکل به صورت زمان-تقسیم محاسبه می‌شود. شش ضرب اول حقیقی محاسبه می‌شوند، سپس مخرج و در نهایت دو تقسیم نهایی:

سیکل	ضرب‌کننده	جمع/ تفریق‌کننده	تقسیم‌کننده	ثبت در لبه بعد
D1	$\text{Re}(a) \times \text{Re}(b)$	بیکار	بیکار	$t_ac \leftarrow ac$
D2	$\text{Im}(a) \times \text{Im}(b)$	بیکار	بیکار	$t_bd \leftarrow bd$
D3	$\text{Re}(a) \times \text{Im}(b)$	$ac + bd$	بیکار	$t_ad \leftarrow ad$ ، $\text{num_re} \leftarrow ac+bd$
D4	$\text{Im}(a) \times \text{Re}(b)$	بیکار	بیکار	$t_bc \leftarrow bc$
D5	$\text{Re}(b) \times \text{Re}(b)$	$bc - ad$	بیکار	$t_cc \leftarrow c^2$ ، $\text{num_im} \leftarrow bc-ad$
D6	$\text{Im}(b) \times \text{Im}(b)$	بیکار	بیکار	$t_dd \leftarrow d^2$
D7	بیکار	$c^2 + d^2$	بیکار	$\text{denom} \leftarrow c^2+d^2$
D8	بیکار	بیکار	$\text{num_re} / \text{denom}$	$\text{Re}(\text{result}) \leftarrow \text{num_re}/\text{denom}$
D9	بیکار	بیکار	$\text{num_im} / \text{denom}$	$\text{Im}(\text{result}) \leftarrow \text{num_im}/\text{denom}$ ، $\text{done}=1$

نکته: چهار سیکل اول تقسیم (D1 تا D4) دقیقاً مشابه چهار سیکل اول ضرب است؛ تنها علامت‌های جمع و تفریق در D3 و D5 با ضرب فرق دارند، چون فرمول تقسیم در صورت کسر + دارد در حالی که ضرب - دارد.

جدول زمان‌بندی مجذور اندازه (۳ سیکل)

سیکل	ضرب‌کننده	جمع/تفریق‌کننده	ثبت در لبه بعد
Q1	$\text{Re}(a) \times \text{Re}(a)$	بیکار	$t_ac \leftarrow \text{Re}^2$
Q2	$\text{Im}(a) \times \text{Im}(a)$	بیکار	$t_bd \leftarrow \text{Im}^2$
Q3	بیکار	$t_ac + t_bd$	$\text{Re}(\text{result}) \leftarrow \text{Re}^2+\text{Im}^2$ ، $\text{Im}(\text{result}) \leftarrow 0$ ، $\text{done}=1$

نتیجه مجذور اندازه کاملاً حقیقی است، بنابراین بخش موهومی خروجی به صراحت صفر می‌شود. این عملیات هیچ سخت‌افزار جدیدی نیاز ندارد و کاملاً از mul و addsub موجود استفاده می‌کند.

۴-۶. فایل inst_fetch.v

حافظه دستورالعمل ۳۲ کلمه‌ای همراه با شمارنده برنامه (PC). با گسترش کد عملیات از ۲ به ۳ بیت، هر دستورالعمل اکنون ۱۸ بیت است: ۳ بیت op، بیت آدرس مقصد، و دو فیلد ۵ بیتی برای آدرس‌های خواندن.

برای اینکه روی برد قابل اجرا باشد، یک بلوک initial \$readmemb به این پیمانه اضافه شده است که برنامه را در زمان کانفیگ FPGA به داخل ROM بار می‌کند. همچنین سیگنال pc_out به عنوان یک پورت خروجی اضافه شده است تا

بتوان مقدار PC را روی LED های سبز نمایش داد:

```
output [A_LEN-1:0] pc_out // برای نمایش روی LED
);
reg [17:0] mem [DEPTH-1:0];
reg [A_LEN-1:0] pc;

initial $readmemb("data/inst_mem.txt", mem);

assign {op, waddr, raddr1, raddr2} = mem[pc];
assign pc_out = pc;

always @(posedge clk or negedge rstN) begin
    if (!rstN) pc <= 0;
    else if (!stall) pc <= pc + 1'b1;
end
```

نکته‌ی مهم: فرمت فایل inst_mem.txt برای سنتز در Quartus باید بدون کاراکترهای زیر خط (_) یا کامنت باشد، چون پارسر Quartus سخت‌گیرتر از ModelSim است.

۴-۷. فایل memory.v

حافظه داده با دو پورت خواندن ناهمزمان (ترکیبی) و یک پورت نوشتن همزمان (با لبه ساعت). با این ساختار، در یک سیکل می‌توان هم دو عملوند جدید را خواند و هم نتیجه‌ی دستور قبلی را نوشت. مشابه inst_fetch، یک بلوک initial برای بارگذاری مقادیر اولیه افزوده شده است.

```
reg `complex mem [DEPTH-1:0];

initial $readmemb("data/initial_mem.txt", mem);

assign rdata1 = mem[raddr1];
assign rdata2 = mem[raddr2];

always @(posedge clk) begin
    if (wen) mem[waddr] <= wdata;
end
```

۴-۸. فایل pipeline.v (با خروجی‌های LED)

این پیمان‌ه MEM، IF، و EX را به هم متصل می‌کند و اکنون دارای خروجی‌های LED برای نمایش وضعیت اجرا روی برد است.

چیدمان LED

خروجی	معنی
LEDR[7:0]	بخش حقیقی نتیجه ALU

خروجي	معني
LEDR[15:8]	بخش موهومي نتيجه ALU
LEDR[16]	مشغول است ALU وقتي — busy
LEDR[17]	پالس يك سيكلي وقتي نتيجه آماده است — alu_done
LEDG[4:0]	شمارنده برنامه (PC)
LEDG[7:5]	كد عمليات جاري (cur_op)
LEDG[8]	وقتي ريست رها شده است روشن — rstN

كد منطق كنترل

```

wire ready_to_issue = !busy || alu_done;
wire stall          = busy && !alu_done;
wire wb_en          = alu_done;

always @(posedge clk or negedge rstN) begin
    if (!rstN) begin
        busy      <= 1'b0;
        alu_start <= 1'b0;
    end else begin
        alu_start <= 1'b0;
        if (ready_to_issue) begin
            cur_op    <= if_op;
            cur_waddr <= if_waddr;
            cur_a      <= rdata1;
            cur_b      <= rdata2;
            alu_start <= 1'b1;
            busy       <= 1'b1;
        end
    end
end

// خروجي هاي LED
assign LEDR[7:0]  = `Re(alu_result);
assign LEDR[15:8] = `Im(alu_result);
assign LEDR[16]   = busy;
assign LEDR[17]   = alu_done;

assign LEDG[4:0]  = if_pc;
assign LEDG[7:5]  = cur_op;
assign LEDG[8]    = rstN;

```


۵-۱. سنتز و کامپایل

از منوی **Processing** گزینه **Start Compilation** را اجرا کنید (یا **Ctrl+L**). بعد از اتمام، گزارش منابع از مسیر زیر قابل مشاهده است:

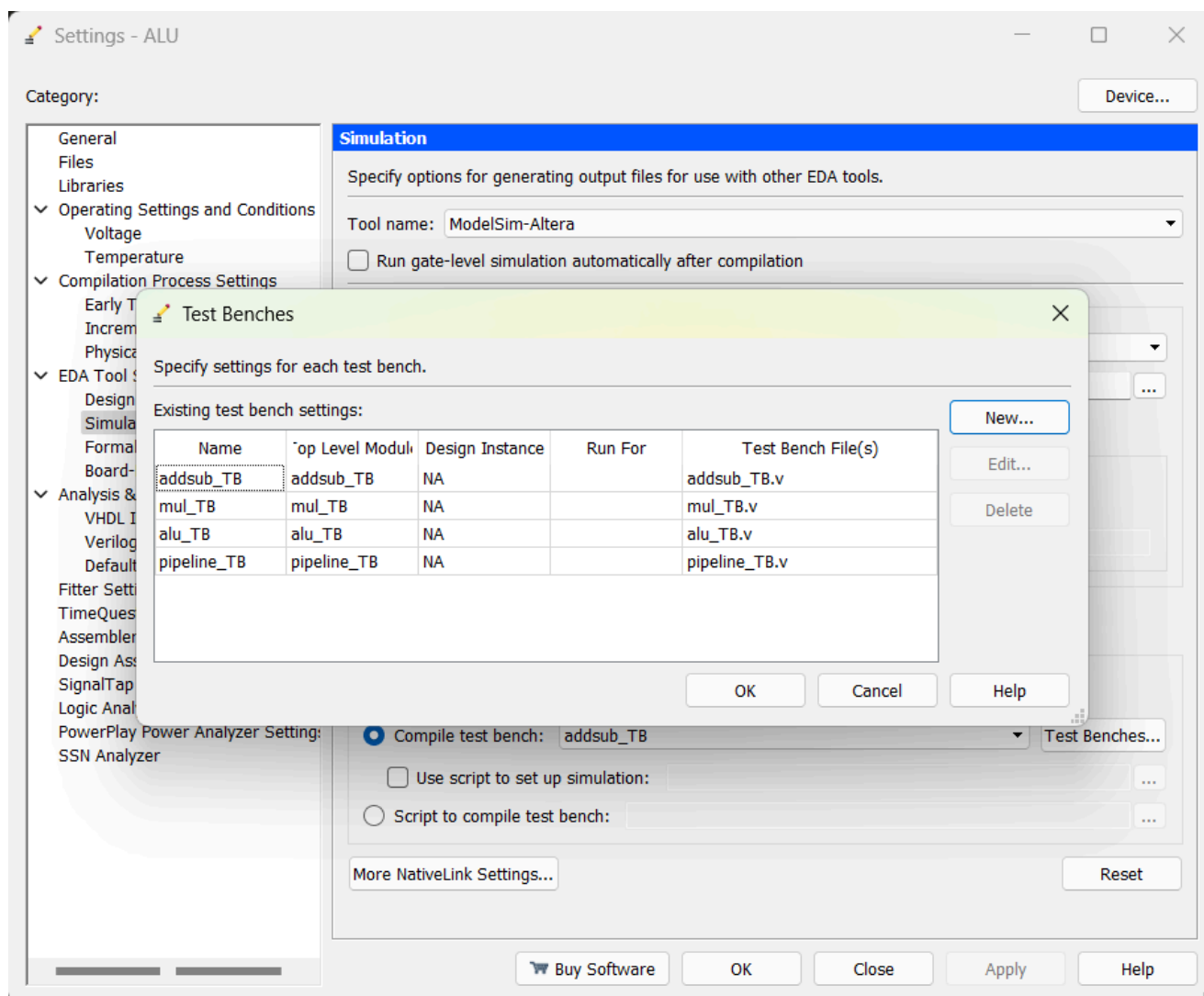
Compilation Report → Analysis & Synthesis → Resource Usage Summary

Flow Summary	
Flow Status	Successful - Thu Jun 04 20:44:51 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	ALU
Top-level Entity Name	pipeline
Family	Cyclone II
Device	EP2C5AF256A7
Timing Models	Final
Total logic elements	0 / 4,608 (0 %)
Total combinational functions	0 / 4,608 (0 %)
Dedicated logic registers	0 / 4,608 (0 %)
Total registers	0
Total pins	2 / 158 (1 %)
Total virtual pins	0
Total memory bits	0 / 119,808 (0 %)
Embedded Multiplier 9-bit elements	0 / 26 (0 %)
Total PLLs	0 / 2 (0 %)

۵-۲. تنظیم تست‌بنچ‌ها در NativeLink

از مسیر Assignments → Settings → EDA Tool Settings → Simulation ...NativeLink settings → Compile test bench → Test Benches کلیک کنید. با دکمه **New** چهار تست‌بنچ را اضافه کنید:

Test bench name	Top level module	File
addsub_TB	addsub_TB	addsub_TB.v
mul_TB	mul_TB	mul_TB.v
alu_TB	alu_TB	alu_TB.v
pipeline_TB	pipeline_TB	pipeline_TB.v



۶. شبیه‌سازی در ModelSim

۶-۱. اجرای شبیه‌سازی

برای اجرای هر تست‌بنچ، در صفحه تنظیمات شبیه‌سازی، آن را از منوی کرکره‌ای **Compile test bench** انتخاب کنید و سپس از منوی **Tools** گزینه **Run Simulation Tool → RTL Simulation** را اجرا کنید.

۶-۲. مدیریت پوشه data

تست‌بنچ pipeline_TB از readmemb\$ برای بارگذاری حافظه دستورالعمل و حافظه داده استفاده می‌کند. ModelSim از مسیر /simulation/modelsim اجرا می‌شود، بنابراین باید پوشه data در همین مسیر قرار گیرد:

```
project_folder/
simulation/
modelsim/
data/
inst_mem.txt
initial_mem.txt
```

برای سنتز در Quartus نیز همین پوشه data باید در ریشه پروژه (کنار فایل .qsf) قرار داشته باشد.

۷. نتایج شبیه‌سازی

۷-۱. تست‌بنچ addsub

این تست‌بنچ تنها رفتار يك جمع/تفریق‌کننده ۸ بیتی حقیقی را بررسی می‌کند:

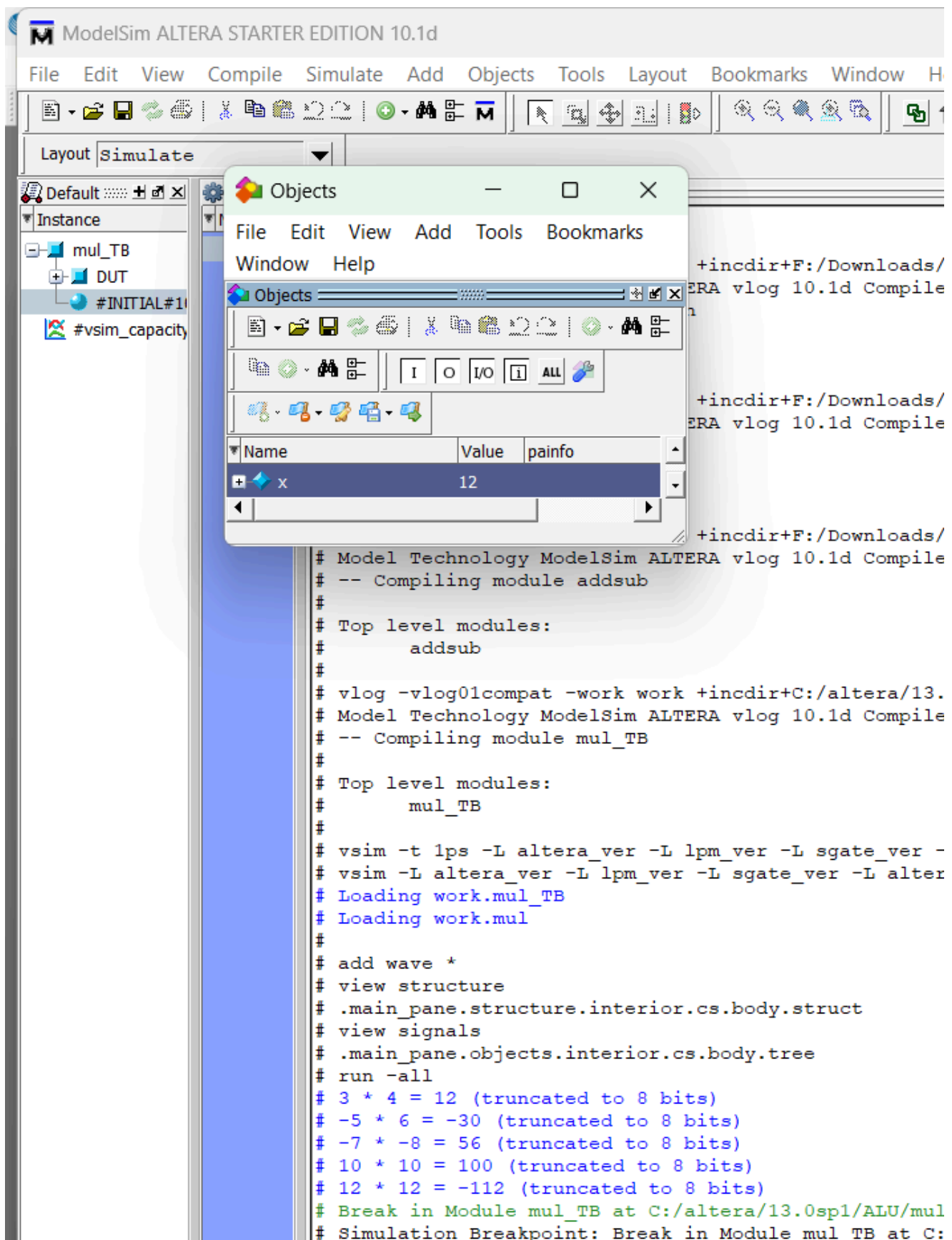
```
10 + 5 = 15
10 - 5 = 5
-20 + 30 = 10
50 - -50 = 100
```

```
# .main_pane.objects.inter
# run -all
# 10 + 5 = 15
# 10 - 5 = 5
# -20 + 30 = 10
# 50 - -50 = 100
# Break in Module addsub_1
# Simulation Breakpoint: F
```

۷-۲. تست‌بنچ mul

در حالت آخر، $12 \times 12 = 144$ از محدوده ۸ بیت علامت‌دار خارج می‌شود و به -112 می‌رسد که پدیده truncation را نشان می‌دهد.

```
3 * 4 = 12
-5 * 6 = -30
-7 * -8 = 56
10 * 10 = 100
12 * 12 = -112 (overflow)
```



۷-۳. تست‌بنج alu

این تست‌بنج مهم‌ترین بخش است: ALU را با تمام پنج عملیات آزمایش می‌کند:

op=000	a=(3,4)	b=(1,2)	=>	(4, 6)
op=001	a=(3,4)	b=(1,2)	=>	(2, 2)
op=010	a=(3,4)	b=(1,2)	=>	(-5, 10)

```

op=011  a=(10,10)  b=(2,0)  =>  (5, 5)
op=011  a=(8,6)    b=(2,0)  =>  (4, 3)
op=011  a=(3,4)    b=(1,2)  =>  (2, 0)
op=100  a=(3,4)                    =>  (25, 0)
op=100  a=(5,0)                    =>  (25, 0)    ←
op=100  a=(6,8)                    =>  (100, 0)

```

۷-۴. تست‌بنج pipeline

این تست‌بنج کامل‌ترین بخش شبیه‌سازی است. یک برنامه شامل ۵ دستور اجرا می‌شود که تمام عملیات‌های جدید را پوشش می‌دهد:

- $\text{mem}[2] \leftarrow \text{mem}[0] + \text{mem}[1] = (3,4) + (1,2) = (4, 6)$
- $\text{mem}[3] \leftarrow \text{mem}[0] - \text{mem}[1] = (3,4) - (1,2) = (2, 2)$
- $\text{mem}[4] \leftarrow \text{mem}[0] \times \text{mem}[1] = (3,4) \times (1,2) = (-5, 10)$
- $\text{mem}[5] \leftarrow \text{mem}[6] / \text{mem}[7] = (10,10) / (2,0) = (5, 5) \leftarrow$ تقسیم
- $\text{mem}[8] \leftarrow |\text{mem}[0]|^2 = |3+4i|^2 = (25, 0) \leftarrow$ مجذور اندازه

خروجی ModelSim:

```

# vsim -t lps -L altera_ver -L lpm_ver -L sgate_ver -L altera_mf_ver -L altera_lnsim_ver -L cycloneii_ver -L rtl_work -L work -voptargs="+acc" pipeline_TB
# vsim -L altera_ver -L lpm_ver -L sgate_ver -L altera_mf_ver -L altera_lnsim_ver -L cycloneii_ver -L rtl_work -L work -voptargs="+acc" -t lps pipeline_TB
# Loading work.pipeline_TB
# Loading work.pipeline
# Loading work.inst_fetch
# Loading work.memory
# Loading work.alu
# Loading work.mul
# Loading work.addsub
# Loading work.divider
# ** Warning: (vsim-3017) C:/altera/13.0sp1/ALU_2/pipeline.v(32): [TFMPC] - Too few port connections. Expected 8, found 7.
#
#      Region: /pipeline_TB/PIPELINE/IF
# ** Warning: (vsim-3722) C:/altera/13.0sp1/ALU_2/pipeline.v(32): [TFMPC] - Missing connection for port 'pc_out'.
#
#
# add wave *
# view structure
# .main_pane.structure.interior.cs.body.struct
# view signals
# .main_pane.objects.interior.cs.body.tree
# run -all
#120 WB mem[2] <= (4, 6) (op=000, a=(3,4), b=(1,2))
#200 WB mem[3] <= (2, 2) (op=001, a=(3,4), b=(1,2))
#340 WB mem[4] <= (-5, 10) (op=010, a=(3,4), b=(1,2))
#560 WB mem[5] <= (5, 5) (op=011, a=(10,10), b=(2,0))
#660 WB mem[8] <= (25, 0) (op=100, a=(3,4), b=(3,4))
#740 WB mem[31] <= (6, 8) (op=000, a=(3,4), b=(3,4))
#820 WB mem[31] <= (6, 8) (op=000, a=(3,4), b=(3,4))
# --- final memory ---
# mem[0] = (3, 4)
# mem[1] = (1, 2)
# mem[2] = (4, 6)
# mem[3] = (2, 2)
# mem[4] = (-5, 10)
# mem[5] = (5, 5)
# mem[6] = (10, 10)
# mem[7] = (2, 0)
# mem[8] = (25, 0)
# Break in Module pipeline_TB at C:/altera/13.0sp1/ALU_2/pipeline_TB.v line 26
# Simulation Breakpoint: Break in Module pipeline_TB at C:/altera/13.0sp1/ALU_2/pipeline_TB.v line 26
# MACRO ./pipeline_run_msim_rtl_verilog.do PAUSED at line 24
# Break key hit

/SIM(pause)>

```

نکته زمان‌بندی قابل توجه: زمان اجرا تا تکمیل اولین دستور (جمع) ۱۲۰ns است، در حالی که دستور تقسیم در ۲۲۰ns بعدی تکمیل می‌شود — این دقیقاً مطابق با تعداد سیکل‌های محاسبه‌شده در FSM است (۱۱ سیکل × ۲۰ns).

۸. پین پلنینگ و دیپلویمنت روی برد DE2

۸-۱. مشخصات برد DE2

برد آلتراي DE2 از FPGA Cyclone II با مدل EP2C35F672C6 استفاده می‌کند. منابع برد که در این پروژه استفاده شده‌اند:

- **CLOCK_50** (PIN_N2) — ۵۰ مگاهرتزي روي برد به عنوان منبع ساعت
- **KEY[0]** (PIN_G26) — براي ريست active-low كليد فشاري دباونس شده با منطق
- **LEDR[17:0]** — قرمز LED عدد ۱۸
- **LEDG[8:0]** — سبز LED عدد ۹

۲-۸. فایل پین‌اسایمنٹ (pipeline_pins.qsf)

برای اجرا روی برد، فایل QSF زیر شامل تمام تخصیص‌های پین و استانداردهای ورودی/خروجی است:

```
set_global_assignment -name FAMILY "Cyclone II"
set_global_assignment -name DEVICE EP2C35F672C6
```

```
set_location_assignment PIN_N2 -to clk
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to clk
```

```
set_location_assignment PIN_G26 -to rstN
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to rstN
```

```
set_location_assignment PIN_AE23 -to LEDR[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTTL" -to LEDR[0]
```

```
set_location_assignment PIN_AE22 -to LEDG[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTTL" -to LEDG[0]
```

نکته مهم: استاندارد I/O براي هر پيښ بايد به صراحت تعريف شود. در غير اين صورت Fitter کوارتوس با خطاي illegal location assignment متوقف می‌شود.

[illegible]

```

# ----- clock -----
set_location_assignment PIN_N2 -to clk
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to clk

# ----- reset (KEY[0] push button) -----
set_location_assignment PIN_G26 -to rstN
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to rstN

# ----- red LEDs -----
set_location_assignment PIN_AE23 -to LEDR[0]
set_location_assignment PIN_AF23 -to LEDR[1]
set_location_assignment PIN_AB21 -to LEDR[2]
set_location_assignment PIN_AC22 -to LEDR[3]
set_location_assignment PIN_AD22 -to LEDR[4]
set_location_assignment PIN_AD23 -to LEDR[5]
set_location_assignment PIN_AD21 -to LEDR[6]
set_location_assignment PIN_AC21 -to LEDR[7]
set_location_assignment PIN_AA14 -to LEDR[8]
set_location_assignment PIN_Y13 -to LEDR[9]
set_location_assignment PIN_AA13 -to LEDR[10]
set_location_assignment PIN_AC14 -to LEDR[11]
set_location_assignment PIN_AD15 -to LEDR[12]
set_location_assignment PIN_AE15 -to LEDR[13]
set_location_assignment PIN_AF13 -to LEDR[14]
set_location_assignment PIN_AE13 -to LEDR[15]
set_location_assignment PIN_AE12 -to LEDR[16]
set_location_assignment PIN_AD12 -to LEDR[17]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[3]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[7]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[8]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[9]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[10]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[11]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[12]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[13]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[14]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[15]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[16]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LEDR[17]

```

۹. نتیجه‌گیری

در این آزمایش، يك ALU چندسیکلي براي اعداد مختلط طراحي و در يك ساختار پایيلاین قرار گرفت. این طراحي علاوه بر عملیات‌هاي پایه (جمع، تفریق، ضرب)، دو عملیات پیشرفته (تقسیم و مجذور اندازه) را نیز پشتیباني مي‌کند، در حالي که محدودیت اصلي صورت آزمایش رعایت شده است.

۹-۱. تحلیل زمان‌بندي

عملیات	سیکل ALU	سرپار صدور	کل
جمع/تفریق مختلط	۲	۱	۳
ضرب مختلط	۵	۱	۶
تقسیم مختلط	۹	۱	۱۰
مجذور اندازه	۳	۱	۴